Bi-Weekly Research Progress Report

Project: IsoShard Proxy: A Distributed CP-Mode Transaction Coordinator & SQL Optimizer on AWS
Proponent: Sarthak Mushrif
Reporting Period: Weeks 3 & 4
Hours Logged: 20 Hours

1. Executive Summary of Progress

During the second reporting period, the primary focus was on finalizing the compiler frontend and building the programmatic structures necessary to interpret the parsed SQL. Building upon the initial lexical analysis from Weeks 1 & 2, the ANTLR4 grammar was expanded to support Data Manipulation Language (DML) write operations. Subsequently, the core `SqlVisitor` pattern was implemented in Java to traverse the generated Abstract Syntax Tree (AST). Finally, an automated testing suite was established using JUnit to validate the parser's accuracy against various SQL edge cases.

2. Detailed Task Breakdown (20 Hours)

- **Grammar Expansion for Write Operations (5 Hours):** Extended the `IsoShardSql.g4` grammar file to include parser rules for `INSERT INTO` and `UPDATE` statements. This involved defining the syntactic structure for `VALUES` tuples and `SET` assignments, which are critical for the Two-Phase Commit (2PC) data mutation phases.
- **AST Visitor Implementation (8 Hours):** Developed the foundational `IsoShardQueryVisitor` Java class. This class extends the auto-generated ANTLR base visitor to programmatically traverse the AST. Implemented specific override methods to extract table names, target columns, and routing keys from the `WHERE` clauses (Predicate Extraction).
- **Unit Testing Framework Setup (5 Hours):** Integrated JUnit 5 into the Maven build lifecycle. Authored a suite of automated unit tests to verify the Lexer and Parser. Tests were designed to pass raw SQL strings (both valid and malformed) into the compiler to ensure syntax errors are gracefully caught and reported without crashing the proxy thread.
- **Code Refactoring & Documentation (2 Hours):** Refactored the grammar to resolve minor ambiguity warnings during ANTLR generation. Documented the AST traversal logic for future integration with the routing engine.

3. Challenges & Roadblocks

- **Type Handling in the Visitor Pattern:** Extracting literals (strings vs. integers) from the AST during traversal presented a type-casting challenge in Java. This was resolved by creating a custom `ValueNode` wrapper class that infers the data type during the Lexing phase, ensuring safe operations when the proxy eventually calculates the routing hash.
- **UPDATE Clause Complexity:** Parsing the `SET column = value` syntax in `UPDATE` queries required careful restructuring of the grammar to ensure the parser didn't confuse assignment operators with comparison operators used in the `WHERE` clause.

4. Objectives for Next Reporting Period (Weeks 5 & 6)

- Implement the **Consistent Hashing Algorithm** (e.g., MurmurHash) in Java to dynamically calculate the target Shard ID based on the extracted AST routing keys.
- Develop a simulated in-memory ”Routing Table” to map calculated hashes to specific AWS RDS node endpoints.
- Begin bootstrapping the foundational **Java Netty** TCP server to accept incoming connections.

Appendix A: Technical Artifact - AST Visitor & Unit Testing

The following Java code snippets demonstrate the traversal logic and testing framework developed during this reporting period.

```java
import org.antlr.v4.runtime.tree.TerminalNode;

public class IsoShardQueryVisitor extends IsoShardSqlBaseVisitor<
    ParsedQueryPlan> {

    @Override
    public ParsedQueryPlan visitSelectStatement(IsoShardSqlParser.
    SelectStatementContext ctx) {
        ParsedQueryPlan plan = new ParsedQueryPlan(QueryType.READ);

        // Extract Table Name
        plan.setTableName(ctx.tableName().getText());

        // Extract WHERE clause for routing (Predicate Pushdown Prep)
        if (ctx.condition() != null) {
            String routingKey = extractRoutingKey(ctx.condition());
            plan.setRoutingKey(routingKey);
        }

        return plan;
    }

    private String extractRoutingKey(IsoShardSqlParser.ConditionContext ctx) {
        // Logic to extract the specific value used in 'WHERE id = X'
        // This will be fed into the Consistent Hashing algorithm next sprint.
        return ctx.expression().getText();
    }
}
```

Listing 1: IsoShardQueryVisitor.java

```java
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class ParserValidationTest {

    @Test
    public void testValidSelectStatement() {
        String sql = "SELECT id, name FROM users WHERE id = 100";
        IsoShardSqlParser parser = CompilerUtils.getParserFor(sql);

        // Ensure no syntax errors were flagged during parsing
        assertEquals(0, parser.getNumberOfSyntaxErrors(), "Parser should not
    throw syntax errors for valid SQL");

        // Verify AST root generation
        IsoShardSqlParser.StatementContext root = parser.statement();
        assertNotNull(root.selectStatement(), "Root should contain a valid
    SELECT context");
    }
}
```

Listing 2: ParserValidationTest.java